

레이다 신호처리 · 표적 위치 산출

좌표계와 좌표변환

해상 함선 레이더 – 안테나 극좌표에서 위경도까지의 변환 체인 설계

2026. 09. 10

발표 대본

표지 · 약 55초

안녕하십니까. 레이더 시스템 소프트웨어 스터디 두 번째 시간, 좌표계와 좌표변환을 발표하겠습니다.

레이더가 표적을 잡으면 신호처리에서 나오는 값은 값은 거리 하나와 각도 둘, 이렇게 세 개뿐입니다. 그런데 전시기에 찍혀야 하는 것은 지도 위의 위도와 경도입니다. 이 둘 사이에는 한 번에 건너될 수 없는 간극이 있습니다.

오늘은 그 간극을 어떤 좌표계로 몇 단계에 걸쳐 메우는지, 그리고 왜 하필 그 좌표계여야 하는지를 먼저 보겠습니다. 그다음 실제 과제 조건으로 손계산까지 따라가고, 마지막에 C 코드와 실행 결과로 확인하겠습니다.

목차

00

좌표변환의 기초

p. 3

왜 좌표계가 여럿인가 · 회전과 평행이동 · 회전행렬의 유도과 성질

01

좌표계 여섯 가지

p. 7

안테나 (극좌표 · 직교좌표 · UV) · 동체 (Body) · INS · NED / ENU · ECEF / ECI · LLA · 변환 체인

02

실전 설계

p. 17

설계 조건 · 단계별 좌표계 선정 · 안테나 → 동체 → 로컬 · 로컬 → ECEF → 위경도 · 역변환

03

C 언어로 구현

p. 23

회전행렬 · 축 맞춤과 회전 순서 · 공식과 코드 대응 · 역변환 · 실행 결과와 UI 확인

2

발표 대본

목차 · 약 1분 00초

전체는 네 부분입니다.

0장에서는 좌표변환이라는 것이 결국 무엇인지 정리합니다. 재료가 회전과 평행이동 두 가지뿐이라는 것, 그리고 회전행렬이 어디서 나오는지까지 봅니다.

1장에서는 이번 설계에 등장하는 좌표계 여섯 가지를 하나씩 봅니다. 각 좌표계의 원점이 어디이고 축이 무엇인지, 그리고 그 단계가 왜 필요한지가 핵심입니다.

2장은 실전 설계입니다. 과제 조건이 좌표계 선택을 어떻게 결정하는지 보고, 입력값 하나를 다섯 단계에 걸쳐 손으로 계산해 위경도까지 끌고 갑니다.

3장은 C 구현입니다. 공식이 코드로 어떻게 옮겨지는지 보고, 실행 결과가 손계산과 맞는지 확인합니다.

00

좌표변환의 기초

- 1 왜 좌표계가 여럿인가
- 2 좌표변환의 기본 동작 - 회전과 평행이동
- 3 회전행렬의 유도과 성질

3

발표 대본

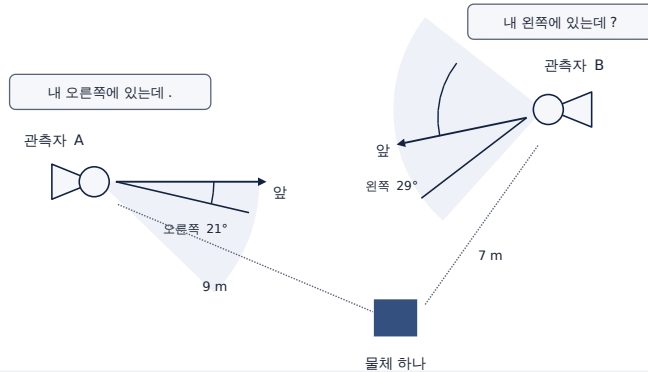
00. 좌표변환의 기초 · 약 20초

먼저 기초부터 정리하겠습니다.

좌표계가 왜 하나로는 안 되는지, 좌표변환이 실제로 하는 일이 무엇인지, 그리고 앞으로 계속 쓰게 될 회전행렬이 어디서 나오고 어떤 성질을 갖는지, 이 세 가지를 보겠습니다.

00. 왜 좌표계가 여럿인가

같은 물체를 두 사람이 본다



관측자 A 가 말하면

"내 오른쪽에 있는데."
앞에서 오른쪽으로 21도, 9 m

관측자 B 가 말하면

"내 왼쪽에 있는데?"
앞에서 왼쪽으로 29도, 7 m

다른 것은 두 가지뿐이다 – 어디에 서서 재는가 (원점), 어디를 보고 재는가 (축)

좌표계란 그 둘을 미리 정해 놓은 약속이고, 기준이 다른 두 좌표계의 숫자를 서로 옮겨 주는 계산이 좌표변환이다.

4

발표 대본

00. 좌표변환의 기초 · 약 1분 25초

아주 단순한 상황부터 보겠습니다. 물체가 하나 있고, 두 사람이 그것을 봅니다.

관측자 A는 "내 오른쪽에 있는데" 라고 말합니다. 앞에서 오른쪽으로 21도, 9미터입니다. 관측자 B는 같은 물체를 보고 "내 왼쪽에 있는데" 라고 합니다. 앞에서 왼쪽으로 29도, 7미터입니다.

물체는 분명히 하나인데 숫자가 완전히 다릅니다. 그런데 누가 틀린 것이 아니라 둘 다 맞습니다.

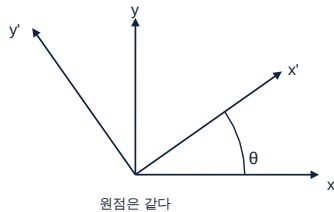
다른 것은 딱 두 가지입니다. 어디에 서서 재는가, 즉 원점. 그리고 어디를 보고 재는가, 즉 축의 방향입니다.

그 둘을 미리 정해 놓은 약속이 좌표계이고, 약속이 다른 두 좌표계의 숫자를 서로 옮겨 주는 계산이 좌표변환입니다.

레이다도 똑같습니다. 안테나는 자기 정면을 기준으로 재고, 배는 뱃머리를 기준으로 알고, 지도는 북극과 그리니치를 기준으로 적습니다. 기준이 서로 다르기 때문에 변환이 필요한 것입니다.

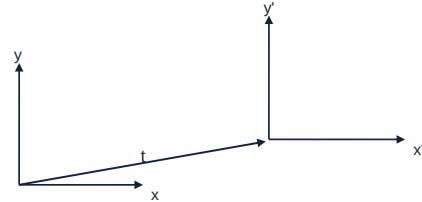
00. 좌표변환의 기본 동작 - 회전과 평행이동

① 회전 (rotation) — 기준 방향이 다를 때

두 좌표계의 축이 어긋난 만큼 θ 돌려서 맞춘다

원점은 같다

② 평행이동 (translation) — 기준 점이 다를 때

두 좌표계의 원점이 떨어진 만큼 t 옮겨서 맞춘다

축방향은 같다

1. 회전 - 방향만 바꾼다

길이도 각도도 변하지 않는다. 축을 돌려 다시 읽을 뿐이다.

2. 평행이동 - 자리만 옮긴다

방향은 그대로다. 원점 사이의 차이를 벡터 하나로 더하거나 뺀다.

$$\text{새 좌표} = \mathbf{R} \cdot \text{기존 좌표} + \mathbf{t}$$

 $\mathbf{R}$ 회전행렬 (3×3) · $\mathbf{t}$ 평행이동 (3×1). 모든 단계는 이 두 개로 동작한다.

5

발표 대본

00. 좌표변환의 기초 · 약 1분 00초

좌표변환의 재료는 두 개뿐입니다.

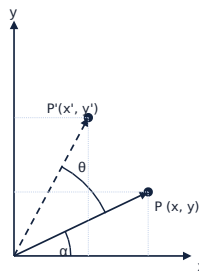
첫째는 회전입니다. 두 좌표계의 원점은 같은데 축이 어긋나 있을 때 씁니다. 어긋난 각도만큼 θ 를 돌려서 맞춥니다. 회전은 방향만 바꿉니다. 길이도, 벡터 사이의 각도도 변하지 않습니다.

둘째는 평행이동입니다. 축 방향은 같은데 원점이 떨어져 있을 때 씁니다. 떨어진 만큼 t 를 더해서 맞춥니다. 이것은 자리만 옮기고 방향은 건드리지 않습니다.

오늘 나올 다섯 단계가 전부 이 둘의 조합입니다. 식으로 쓰면 "새 좌표 = $\mathbf{R}$ 곱하기 기존 좌표, 더하기 $\mathbf{t}$ " 한 줄이고, 단계마다 그 $\mathbf{R}$ 과 $\mathbf{t}$ 를 누가 공급하느냐만 달라집니다.

00. 회전행렬의 유도와 성질

삼각함수 덧셈정리 한 줄에서 나온다.



원래 점을 극좌표로 쓰면

$$x = r \cdot \cos \alpha, \quad y = r \cdot \sin \alpha$$

θ 만큼 더 돌리면 각이 $\alpha + \theta$ 가 되므로

$$x' = r \cdot \cos(\alpha + \theta) = r \cdot \cos \alpha \cdot \cos \theta - r \cdot \sin \alpha \cdot \sin \theta$$

$$y' = r \cdot \sin(\alpha + \theta) = r \cdot \sin \alpha \cdot \cos \theta + r \cdot \cos \alpha \cdot \sin \theta$$

$r \cdot \cos \alpha = x$, $r \cdot \sin \alpha = y$ 를 되돌려 넣으면

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

이 두 줄을 표로 적은 것이 $R_z(\theta)$ 다.

세 축 둘레의 회전행렬

 $R_z(\theta)$ — z 축 둘레 (yaw)

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

 $R_y(\theta)$ — y 축 둘레 (pitch)

$$\begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

 $R_x(\theta)$ — x 축 둘레 (roll)

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

성질 — 역변환에서 전치를 쓰는 근거

$R^T \cdot R = I$, 즉 $R^{-1} = R^T$ — 역행렬을 따로 구할 필요가 없다 (p.22 역변환)

$\det R = 1$ — 오른손 좌표계는 오른손 좌표계로 남는다

$|Rp| = |p|$ — 길어도, 두 벡터가 이루는 각도도 그대로다

위 유도는 점을 θ 돌린 것. 축을 θ 돌려 같은 점을 다시 읽으면 R^T 가 된다.

6

발표 대본

00. 좌표변환의 기초 · 약 1분 45초

회전행렬은 외우는 것이 아니라 유도되는 것입니다.

왼쪽 그림을 보시면, 점 P를 극좌표로 쓰면 x 는 r 코사인 α , y 는 r 사인 α 입니다. 여기서 θ 만큼 더 돌리면 각이 α 더하기 θ 가 되니까, x' 은 r 코사인 α 더하기 θ 입니다. 삼각함수 덧셈정리로 풀면 r 코사인 α 코사인 θ 빼기 r 사인 α 사인 θ 가 됩니다.

그런데 r 코사인 α 가 원래 x 이고 r 사인 α 가 원래 y 입니다. 그래서 x' 은 x 코사인 θ 빼기 y 사인 θ 가 됩니다. y' 도 같은 방식으로 나옵니다. 이 두 줄을 행렬로 묶은 것이 회전행렬입니다.

이것을 세 축 둘레로 각각 만든 것이 오른쪽 R_z , R_y , R_x 입니다. 각각 yaw, pitch, roll에 해당합니다. 보시면 R_y 만 부호 배치가 다른데, 뒤에 코드에서 다시 짚겠습니다.

그리고 오늘 가장 많이 쓰게 될 성질이 맨 아래에 있습니다. R 전치 곱하기 R 이 단위행렬입니다. 즉 역행렬이 곧 전치입니다. 역행렬을 따로 구할 필요가 없다는 뜻이고, 뒤에 나올 역변환 전체가 이 한 줄에 기대고 있습니다.

01

좌표계 여섯 가지

- 1 안테나 좌표계 - 극좌표 · 직교좌표 · UV
- 2 동체 (Body) 좌표계
- 3 INS 좌표계
- 4 NED / ENU 국지수평 좌표계
- 5 ECEF / ECI 지구 중심 좌표계
- 6 LLA 측지 좌표계
- 7 변환 체인

7

발표 대본

01. 좌표계 여섯 가지 · 약 35초

이제 이번 설계에 실제로 등장하는 좌표계들을 보겠습니다.

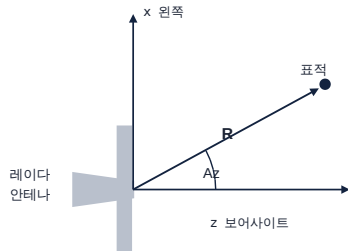
안테나에서 출발해서, 배에 붙어 있는 동체 좌표계, 자세를 공급하는 INS, 배 주변의 수평면인 NED, 지구 전체 기준인 ECEF, 그리고 마지막 출력인 위경도까지 순서대로 봅니다.

각 좌표계마다 원점이 어디이고 축이 무엇인지, 그리고 그 단계가 왜 필요한지에 초점을 맞추겠습니다.

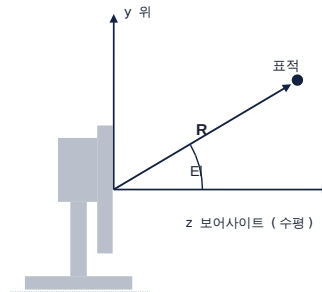
01. 안테나 좌표계 ① 극좌표 (Polar)

레이다가 물리적으로 잴 수 있는 것 - 거리 하나와 각도 둘. 기준은 안테나 자기 정면 (보어사이트).

위에서 본 안테나 - 방위각 Az(Azimuth)



옆에서 본 안테나 - 고각 EI(Elevation)



세 성분

R 시선거리

Range · 전파 왕복시간 $\times c + 2$

Az 방위각

Azimuth · 보어사이트에서 왼쪽 (반시계) 으로

EI 고각

Elevation · 수평면에서 위로

20 km / 30° / 0°

보어사이트 = 안테나가 똑바로 바라보는 방향. 안테나 신호가 가장 세게 나가고 들어오는 한가운데 방향.

8

발표 대본

01. 좌표계 여섯 가지 · 약 1분 05초

변환 체인의 출발점입니다. 레이다가 물리적으로 잴 수 있는 것은 세 개뿐입니다.

거리 R 은 전파를 쏘고 되돌아오는 시간에 빛의 속도를 곱하고 2 로 나눈 값입니다. 왕복이기 때문에 2 로 나눕니다.

방위각과 고각은 안테나 정면, 즉 보어사이트를 기준으로 잰 각입니다. 보어사이트는 안테나 신호가 가장 세게 나가고 들어오는 한가운데 방향을 말합니다.

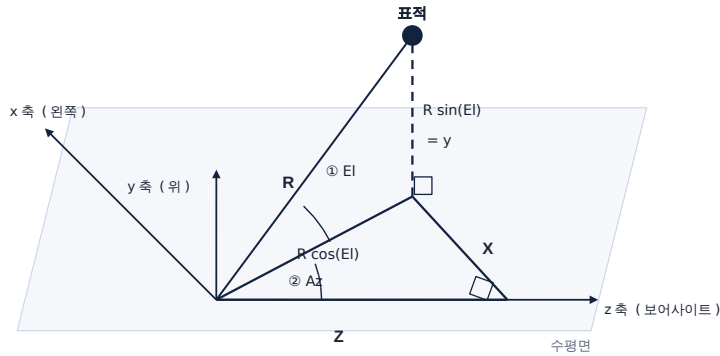
위 그림은 안테나를 위에서 내려다본 것으로 좌우 방향인 방위각을, 아래 그림은 옆에서 본 것으로 위아래 방향인 고각을 나타냅니다.

이번 과제의 입력은 20 킬로미터, 방위 30도, 고각 0도입니다. 오늘 다루는 모든 계산이 이 세 숫자에서 출발합니다.

01. 안테나 좌표계 ② 직교좌표 (Cartesian)

같은 표적을 각도 대신 서로 직각인 세 축 방향 거리 (x, y, z) 로 적은 것.

z 보어사이트 · x 왼쪽 · y 위



예제값 $R = 20,000 \text{ m}$ · $Az = 30^\circ$ · $EI = 0^\circ \rightarrow x = +10,000.0000 \text{ m}$, $y = 0.0000 \text{ m}$, $z = +17,320.5081 \text{ m} (= 10,000\sqrt{3})$

왜 직교좌표로 바꾸나

극좌표끼리는 회전을 합성할 수 없다.
회전행렬은 벡터에만 곱할 수 있다.

공식

$$x = R \times \cos(EI) \times \sin(Az)$$

$$y = R \times \sin(EI)$$

$$z = R \times \cos(EI) \times \cos(Az)$$

9

발표 대본

01. 좌표계 여섯 가지 · 약 1분 30초

극좌표는 사람이 읽기에는 편하지만, 그대로 회전시킬 수는 없습니다. 회전행렬은 벡터에만 곱할 수 있기 때문입니다. 각도끼리 더한다고 회전이 합성되지 않습니다.

그래서 같은 표적을 서로 직각인 세 축 방향의 거리로 다시 적습니다. 안테나 좌표계는 z가 보어사이트, x가 왼쪽, y가 위입니다.

공식은 그림에서 그대로 나옵니다. 먼저 고각으로 위아래 성분을 뺍니다. y가 R 사인 EI입니다. 그리고 남은 R 코사인 EI이 수평면에 깔린 길이인데, 이것을 다시 방위각으로 나누어 x는 사인 Az를, z는 코사인 Az를 곱합니다.

이번 값을 넣어 보겠습니다. 고각이 0도라 y는 0입니다. x는 2만 곱하기 사인 30도로 정확히 1만 미터, z는 2만 곱하기 코사인 30도로 1만 7320.51 미터입니다.

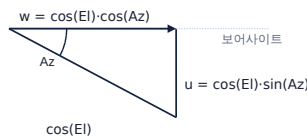
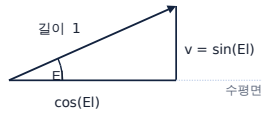
여기서 중요한 것은 표현만 바뀌었지 표적은 그대로라는 점입니다. 아직 회전도 이동도 하지 않았습니다.

01. 안테나 좌표계 ③ UV(방향코사인)

표적 방향을 길이 1 인 화살표로 보고, 그 화살표를 안테나 세 축에 나눠 담은 값이다.

길이 1 화살표를 세 축에 나눠 담기

① 티 로 위아래를 떼고 → ② 남은 cos(EI) 을 Az 로 나눈다



$$u = \cos(EI) \cdot \sin(Az) \quad v = \sin(EI) \quad w = \cos(EI) \cdot \cos(Az)$$

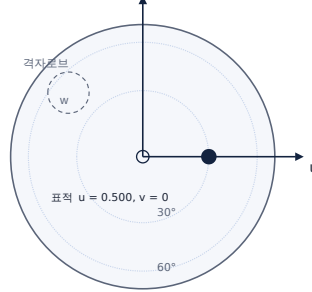
세 성분을 제곱해 더하면 1 이 된다 — 길이 1 인 화살표를 나눠 담은 것이므로.

각 성분은 그 축과 이루는 각의 코사인이라 '방향코사인' 이라 부른다. u, v 는 약자가 아니라 성분의 이름이다.

* UV 에는 거리가 없다 → 방향만 담은 표현이라, 표적 위치를 옮기는 이번 변환에는 들어가지 않는다.

안테나 면에서 본 u - v 평면

점 (u, v) 는 표적 방향의 그림자



$$u^2 + v^2 \leq 1 \text{ — 이 원 밖으로는 빔을 못 만든다}$$

왜 UV 를 쓰나

현대 레이더는 대부분 위상배열이다.
수백 ~ 수천 개 소자에 위상차를 줘서
빔 방향을 전자적으로 바꾼다.

$$\phi(m, n) = -k (m \cdot d \cdot u + n \cdot d \cdot v)$$

$k = 2\pi/\lambda \cdot d$ 소자 간격 · m, n 소자 번호
소자 번호에 u, v 를 곱하기만 하면 된다.
각도 (Az, EI) 로는 이렇게 단순해지지 않는다.
격자로브 : $d > \lambda/2$ 이면 원치 않는 제 2 빔이 생긴다.
UV 에서 각도로 되돌리면
 $Az = \text{atan2}(u, w)$
 $EI = \text{atan2}(v, \sqrt{u^2 + w^2})$

10

발표 대본

01. 좌표계 여섯 가지 · 약 1분 45초

안테나 좌표계의 세 번째 표현인 UV, 방향코사인입니다.

표적 방향을 길이 1 인 화살표로 보고, 그 화살표를 안테나 세 축에 나눠 담은 값입니다. 만드는 순서는 직교좌표와 같습니다. 먼저 고각으로 위아래를 떼면 v 가 사인 티 이 되고, 남은 코사인 티 을 방위각으로 나누면 u 와 w 가 나옵니다. 세 성분을 제곱해서 더하면 1 이 되는데, 길이 1 인 화살표를 쪼갠 것이니 당연한 결과입니다.

가운데 그림은 안테나 면에서 본 u-v 평면으로, 표적 방향의 그림자라고 보시면 됩니다. u 제곱 더하기 v 제곱이 1 이하인 원 안쪽에서만 빔을 만들 수 있습니다.

왜 이런 표현을 쓰는지가 오른쪽에 있습니다. 현대 레이더는 대부분 위상배열입니다. 수백에서 수천 개의 소자에 위상차를 줘서 빔 방향을 전자적으로 바꾸는데, 그 위상차 공식을 보시면 소자 번호에 u 와 v 를 곱하기만 하면 됩니다. 각도 Az, TI 로는 이렇게 단순해지지 않습니다.

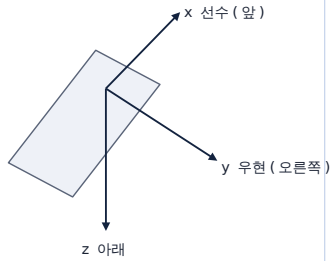
다만 오늘 체인에는 UV 가 들어가지 않습니다. UV 에는 거리가 없기 때문입니다. 방향만 담은 표현이라 표적의 위치를 옮기는 이번 변환에는 쓸 수 없습니다.

01. 동체 (Body) 좌표계

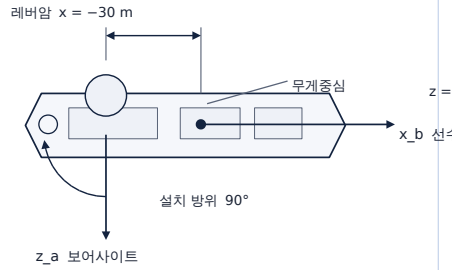
동체 (Body) 좌표계 = FRD

FRD = Forward-Right-Down.

원점은 무게중심. 세 축은 배에 붙어 함께 움직인다.

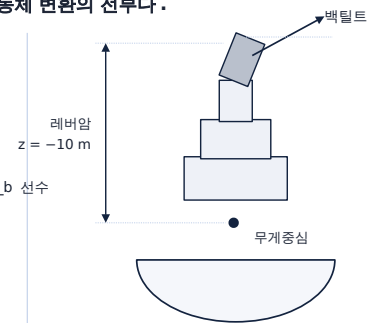


설치 방위 · 백틸트 · 레버암 - 이 셋이 안테나 → 동체 변환의 전부다.



위에서 본 그림

이번 과제 값 : 설치 방위 90° · 백틸트 0°



선미에서 본 그림

레버암 (-30, 0, -10) m

원점 · 축

원점 = 배의 무게중심

x 선수 · y 우현 · z 아래

→ FRD(Forward-Right-Down)

안테나 → 동체 변환

p 안테나 (x 왼쪽, y 위, z 보어사이트) → 축 맞춤 A → Ry(백틸트) → Rz(설치방위) → + 레버암 → p 동체

이번 과제 값 →

Rz(-90°)·Rx(-90°)

백틸트 0°

설치 방위 90°

레버암 (-30, 0, -10) m

식으로 쓰면 p 동체 = Rz(설치방위) · Ry(백틸트) · A · p 안테나 + 레버암
 $A = Rz(-90°) \cdot Rx(-90°)$ → 안테나 면 기준 축을 동체 축으로 돌려 놓는 상수 행렬 (참고자료 : CIWS-II 데이터처리를 위한 좌표계 및 좌표변환 §2.2.9)

발표 대본

01. 좌표계 여섯 가지 · 약 1분 40초

안테나에서 나온 값을 이제 배 기준으로 옮깁니다.

동체 좌표계는 FRD 입니다. Forward, Right, Down. x 가 선수 즉 앞, y 가 우현 즉 오른쪽, z 가 아래입니다. 원점은 배의 무게중심이고, 세 축은 배에 붙어서 배와 함께 움직입니다.

안테나에서 동체로 가는 데 필요한 정보는 세 가지뿐이고, 전부 설치 도면에서 나옵니다.

첫째는 설치 방위입니다. 안테나를 뱃머리 기준으로 어느 방향에 달았는가. 이번 과제는 90도, 즉 우현을 봅니다.

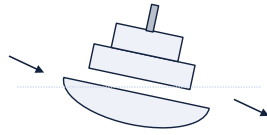
둘째는 백틸트입니다. 안테나 면을 위로 얼마나 들어 올렸는가. 이번 과제는 0도입니다.

셋째는 레버암입니다. 무게중심에서 안테나까지의 거리 벡터로, 이번 과제는 (-30, 0, -10) 미터입니다. 선수 방향으로 30미터 뒤, 그리고 10미터 위라는 뜻입니다.

아래 흐름도가 이 셋이 어떻게 들어가는지 보여줍니다. 축 맞춤 A 를 먼저 걸고, 백틸트, 설치 방위 순으로 회전한 다음, 마지막에 레버암을 더합니다. 이 순서가 왜 이래야 하는지는 3장에서 따로 다루겠습니다.

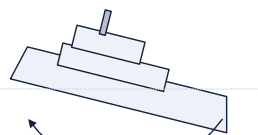
01. INS 좌표계

자세각 = 동체 축이 북 · 동 · 아래에 대해 얼마나 돌아가 있다. 동체 좌표계는 북이 어딘지 모르지만, INS 는 안다.



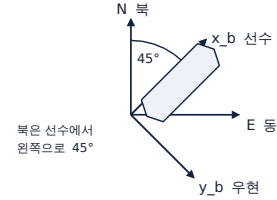
roll (횡동요)

뒤에서 본 그림 · x 축 (선수축) 둘레 · 우현이 내려가면 +
이번 과제 0°



pitch (종동요)

옆에서 본 그림 · y 축 (우현축) 둘레 · 선수가 올라가면 +
이번 과제 0°



yaw (선수방위)

위에서 본 그림 · z 축 (아래축) 둘레 · 진북에서 시계방향 +
이번 과제 45°

INS 좌표계란

INS 센서에 붙은 좌표계 - 원점은 센서 블록의 기준점, 축은 센서의 직교 3 축
배의 자세와 위치를 재서 준다

INS 가 주는 값

자세	roll 0° · pitch 0° · yaw 45°
위치	36.408 °N · 127.307 °E · 0 m
속도	이번 과제에선 사용 안 함

INS 는 값의 공급원이다. - 자세각은 로컬 NED 좌표를 만드는데, 위경도는 그 뒤 지구 좌표로 갈 때 쓰인다.

yaw 45° 는 "진북이 선수에서 왼쪽으로 45°" 라는 뜻이다. 북이 어느 쪽인지 알았으니, 동체 좌표를 북 · 동 기준으로 다시 적을 수 있다.

12

발표 대본

01. 좌표계 여섯 가지 · 약 1분 30초

동체 좌표계에는 결정적인 한계가 있습니다. 북쪽이 어딘지 모릅니다. 배에 붙어 있으니 배가 돌면 축도 같이 돌아 버립니다.

그 정보를 주는 것이 INS, 관성항법장치입니다. INS 는 센서 블록에 붙은 자기 좌표계를 갖고 있고, 보통 무게중심에 축을 맞춰 설치합니다.

INS 가 주는 값은 자세와 위치입니다. 자세는 롤, 피치, 요 세 각도입니다. 롤은 선수축 둘레로 배가 좌우로 기우는 것, 피치는 우현축 둘레로 뱃머리가 들리는 것, 요는 아래축 둘레로 배가 어느 방향을 보고 있는가입니다.

이번 과제는 롤 0도, 피치 0도, 요 45도입니다. 요 45도는 배가 진북에서 시계방향으로 45도 돌아가 있다는 뜻이고, 바꿔 말하면 진북이 선수에서 왼쪽으로 45도에 있다는 것입니다.

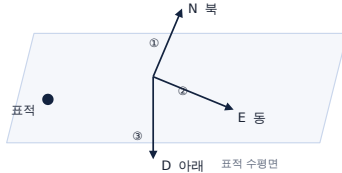
위치는 북위 36.408도, 동경 127.307도, 고도 0 입니다. 자세각은 다음 단계에서 NED 로 갈 때 쓰이고, 위경도는 그다음 지구 좌표로 갈 때 쓰입니다.

01. NED / ENU 국지수평 좌표계

배가 아무리 흔들려도 북쪽은 계속 북쪽이다.

NED (항공우주 · 항법 · 무기체계)

North-East-Down, 북 → 동 → 아래 순서, 셋째 축이 아래로 +



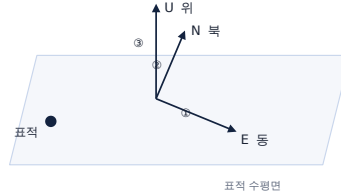
(N -5,197.59, E +19,297.30, D -10.00) m

표적 높이는 수평면 기준

북 · 동은 같은 방향이다. 셋째 축만 뒤집히고, 둘 다 오른손 좌표계다.

ENU (측지 · 측량 · GIS · 로봇틱스)

East-North-Up, 동 → 북 → 위 순서, 셋째 축이 위로 +



(E +19,297.30, N -5,197.59, U +10.00) m

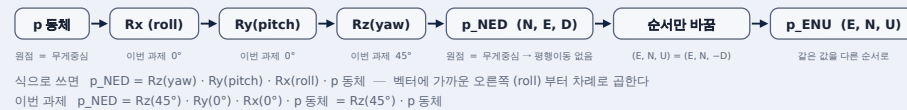
이 좌표계의 존재 이유

배와 함께 이동하지만
함께 회전하지는 않는다.

그래서 표적이 실제로 움직인 것과
배가 흔들린 것을 구분할 수 있다.

추적 필터를 여기서 돌리는 이유다.

동체 → NED 변환 공식



13

발표 대본

01. 좌표계 여섯 가지 · 약 1분 40초

배가 아무리 흔들려도 북쪽은 계속 북쪽입니다. 그 성질을 쓰는 좌표계가 NED 입니다.

NED 는 North, East, Down 으로 북, 동, 아래 순서이고 셋째 축이 아래로 양수입니다. ENU 는 East, North, Up 으로 동, 북, 위 순서입니다. 두 그림을 비교해 보시면 북과 동은 같은 방향이고 셋째 축만 뒤집혀 있습니다. 둘 다 오른손 좌표계이고 정보량이 같습니다. 항공우주와 무기체계에서는 관례상 NED 를, 측지나 GIS 쪽에서는 ENU 를 씁니다.

이 좌표계가 왜 필요한지가 오른쪽에 있습니다. NED 는 배와 함께 이동하지만 배와 함께 회전하지는 않습니다. 그래서 표적이 실제로 움직인 것과 배가 흔들린 것을 구분할 수 있습니다. 추적 필터를 이 좌표계에서 돌리는 이유입니다.

아래가 동체에서 NED 로 가는 변환입니다. 원점이 둘 다 무게중심이라 평행이동이 없고 회전만 걸립니다. 롤, 피치, 요 세 회전을 곱하는데 벡터에 가까운 롤부터 차례로 걸립니다. 이번 과제는 롤과 피치가 0도라 결국 요 45도만 남습니다.

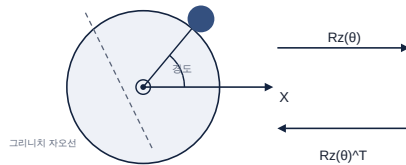
01. ECEF / ECI 지구 중심 좌표계

둘 다 원점은 같고, 축이 도느냐 마느냐만 다르다.

ECEF — 지구와 함께 돈다

Earth-Centered, Earth-Fixed

축과 건물이 같이 도니 사이 각(경도)이 언제 봐도 같다.



X 축 = 그리니치 자오선. 지구와 함께 돈다.

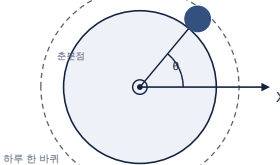
θ = GMST (Greenwich Mean Sidereal Time). 시각이 1 ms 어긋나면 적도에서 0.47 m 어긋난다.

변환 공식

ECI — 별에 고정, 돌지 않는다

Earth-Centered Inertial

축이 멈춰 있으니 건물은 하루에 한 바퀴 돈다.



X 축 = 별 (준분점) 에 고정 . 돌지 않는다 .

**ECEF — Earth-Centered, Earth-Fixed (이번
과제 0)**

원점 = 지구 중심 . X = 그리니치 자오선 . Z = 북극 . Y = 오른손

지구와 함께 돈다 → 지표 한 점의 좌표가 시각과 무관
GPS · 지도 · 위경도의 바탕이 되는 좌표계

ECI — Earth-Centered Inertial (이번 과제 X)

원점 = 지구 중심 · X = 춘분점 (별) 방향 · Z = 북극

돌지 않는다 → 뉴턴 법칙이 그대로 성립

위성 · 탄도탄의 궤도 계산에 쓴다 (중력만 받는 운동)



줄임말 : $R_z(\text{경도}) \cdot R_y(-90^\circ - \text{위도}) = C_{\text{ecef} \leftarrow \text{ned}}$
 R_N 은 그 위도의 곡률반경, e^2 는 WGS-84 상수 - 다음 장 LLA 에서

P 배 = 배의 위경도 · 고도 (φ, λ, h) 에서
 $X = (R_N + h)\cos\phi\cos\lambda$, $Y = (R_N + h)\cos\phi\sin\lambda$, $Z = (R_N(1 - e^2) + h)\sin\phi$

14

발표 대본

01. 좌표계 여섯 가지 · 약 1분 45초

여기서부터는 기준이 배가 아니라 지구 전체입니다.

ECEF 와 ECI 는 원점이 지구 중심으로 똑같습니다. 다른 것은 축이 도느냐 마느냐뿐입니다.

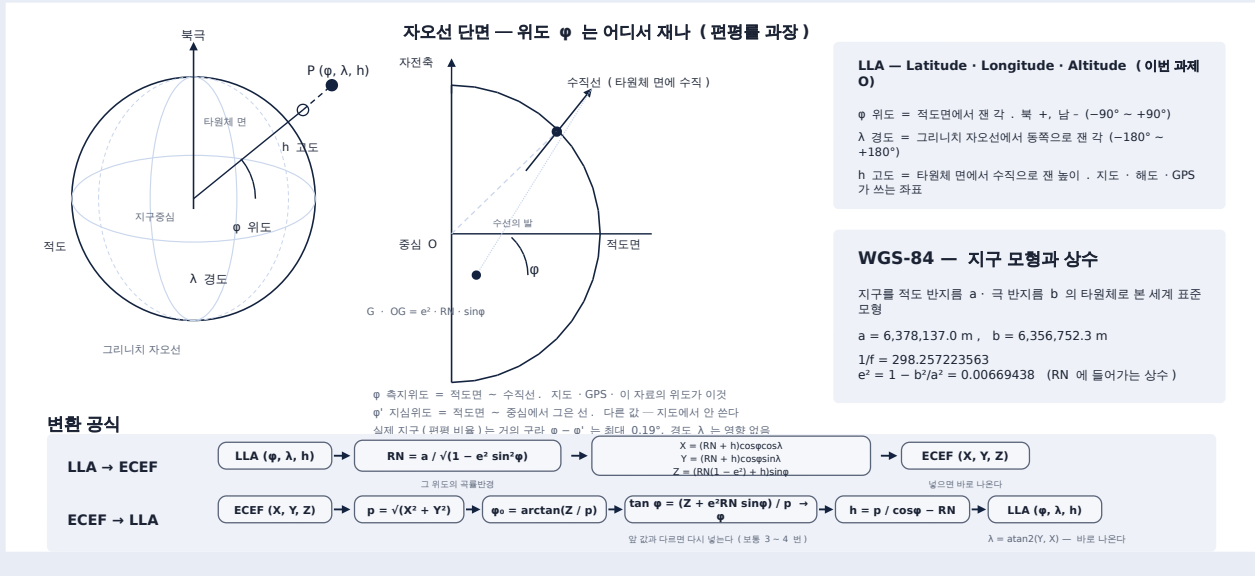
ECEF 는 Earth-Centered, Earth-Fixed 입니다. X 축이 그리니치 자오선을 향하고 지구와 함께 돕니다. 축과 지표의 건물이 같이 도니까, 지표 한 점의 좌표가 시간이 지나도 변하지 않습니다.

ECI는 Earth-Centered Inertial 입니다. X 축이 춘분점, 즉 별에 고정되어 있어서 돌지 않습니다. 그래서 지표의 건물이 하루에 한 바퀴 돕니다. 대신 축이 관성계라 뉴턴 법칙이 그대로 성립하고, 위성이나 탄도탄처럼 궤도를 전파해야 할 때 씁니다.

둘 사이 변환은 지구 자전각 θ 하나로 끝나는데, 이 θ 가 GMST, 즉 시각으로 정해집니다. 그래서 시각이 1밀리초만 어긋나도 적도에서 0.47미터가 어긋납니다.

이번 과제는 지표에 있는 표적을 다루므로 ECEF 만 씁니다. 아래 변환을 보시면 NED 를 지구 축에 맞춰 두 번 돌린 다음 배의 ECEF 위치를 더합니다. 회전은 배의 위경도가, 이동은 배의 위치가 공급합니다.

01. LLA 측지 좌표계



15

발표 대본

01. 좌표계 여섯 가지 · 약 1분 40초

마지막 좌표계이자 최종 출력입니다. 위도, 경도, 고도.

여기서 처음으로 지구의 모양이 계산에 직접 들어옵니다. 지구는 완전한 구가 아니라 적도 쪽이 볼록하고 극 쪽이 눌린 타원체입니다. 그 모양을 정한 세계 표준이 WGS-84 이고, 적도 반지름이 6,378,137 미터, 극 반지름이 6,356,752 미터입니다. 차이가 약 21 킬로미터, 비율로는 0.3 퍼센트 정도입니다.

가운데 그림이 중요합니다. 위도를 어디서 재느냐인데, 우리가 쓰는 측지위도 ϕ 는 타원체 면에 수직인 선이 적도면과 이루는 각입니다. 지구 중심에서 그은 선으로 재는 지심위도와는 다른 값이고, 지도와 GPS 가 쓰는 것은 측지위도입니다.

아래 변환을 보시면, LLA 에서 ECEF 로는 공식에 넣으면 한 번에 나옵니다. 그런데 반대 방향은 그렇지 않습니다. 곡률반경 RN 을 구하려면 위도를 알아야 하고, 위도를 구하려면 RN 을 알아야 하는 순환 구조입니다. 그래서 초기 값을 넣고 값이 더 안 변할 때까지 반복해서 풉니다. 보통 서너 번이면 수렴합니다.

01. 변환 체인

회전은 설치 방위와 INS 자세가, 이동은 레버암과 INS 위치가 공급한다 — 그대로 오차의 출처 목록이 된다.

신호처리 출력

R, Az, El (안테나 극좌표)

극좌표를 x, y, z 세 성분으로 분해

표현만 바꿀 뿐 · 회전도 이동도 없음

안테나 좌표계

Polar → Cartesian → (UV)

$p_{\text{동체}} = R_z(\text{설치방위}) \cdot R_y(\text{백틸트}) \cdot A \cdot p_{\text{안테나}} + \text{레버암}$

회전 + 이동 · 설치 도면이 공급 · 고정형이라 상수 · A 는 축 맞춤 상수 행렬

동체 (Body) 좌표계

x= 선수 y= 우현 z= 아래 (FRD)

$C_{\text{ned}} \leftarrow \text{body} = R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$

회전 · INS 자세가 공급 · 매 순간 변환 · 벡터에 가까운 Rx(roll) 부터 적용된다

로컬 (NED) 좌표계

x= 북 y= 동 z= 아래 데이터처리

$p_{\text{ECEF}} = C_{\text{ecef} \leftarrow \text{ned}}(\text{위도}, \text{경도}) \cdot p_{\text{NED}} + \text{배의 ECEF 위치}$

회전 + 이동 · INS 위치가 공급

ECEF 좌표계

지구중심 지구고정 직교좌표

위도 · 고도 반복 계산 (값이 안 변할 때까지)

WGS-84 타원체 직교좌표 → 위경도

LLA 좌표계

위도 경도 고도 → 전시기

16

발표 대본

01. 좌표계 여섯 가지 · 약 1분 25초

지금까지 본 좌표계를 하나로 이어 붙인 것이 이 표입니다. 위에서 아래로 다섯 단계입니다.

첫째, 극좌표를 직교좌표로 바꿉니다. 표현만 바뀌고 회전도 이동도 없습니다.

둘째, 안테나에서 동체로 갑니다. 회전과 이동이 모두 있고 설치 도면이 값을 공급합니다. 고정형 안테나라 이 값들은 상수입니다.

셋째, 동체에서 NED 로 갑니다. 원점이 같아서 회전만 있고 INS 자세가 공급합니다. 이 값은 매 순간 변환합니다.

넷째, NED 에서 ECEF 로 갑니다. 회전과 이동이 모두 있고 INS 위치가 공급합니다.

다섯째, ECEF 를 위경도로 바꿉니다. 여기만 반복 계산입니다.

오른쪽 열을 보시면 각 단계의 회전과 이동을 누가 공급하는지가 적혀 있습니다. 이 표는 그대로 오차의 출처 목록이 됩니다. 설치 도면이 틀리면 2단계가, INS 자세가 흔들리면 3단계가, INS 위치가 부정확하면 4단계가 들어갑니다.

실전 설계

- 1 설계 조건
- 2 단계별 좌표계 선정
- 3 단계별 변환 – 안테나 → 동체 → 로컬
- 4 단계별 변환 – 로컬 → ECEF → 위경도
- 5 역변환

17

발표 대본

02. 실전 설계 · 약 25초

이제 실제 과제 조건으로 설계를 해 보겠습니다.

주어진 조건이 좌표계 선택을 어떻게 결정하는지 보고, 왜 어떤 좌표계는 쓰지 않는지도 함께 정리하겠습니다. 그다음 입력값 하나를 실제로 손으로 계산해서 위경도까지 끌고 가 보겠습니다.

02. 설계 조건

주어진 조건 여섯 가지와, 그것이 설계를 어떻게 묶는가

조건	내용	설계에 미치는 영향
① 플랫폼	해상 함선, 고정형 안테나	안테나 → 동체 변환이 상수. 배의 흔들림은 INS 자세로 보상
② 안테나 설치	선수 기준 시계방향 90°, 무게중심에서 30m 뒤 / 10m 위	설치 방위 90°, 백틸트 0°, 레버암 (-30, 0, -10) m
③ INS 설치	무게중심과 축 일치	INS 값 (자세 · 위치) 을 보정 없이 그대로 동체 값으로 쓴다
④ 입력	안테나 기준 거리 · 방위각 · 고각	체인의 출발점 = 안테나 극좌표
⑤ 처리	로컬 좌표계에서 수행	체인의 중간 목적지 = 로컬 수평 좌표계 (NED)
⑥ 출력	위 / 경 / 고도 + 안테나 극좌표	정변환뿐 아니라 역변환도 필요

18

발표 대본

02. 실전 설계 · 약 40초

주어진 조건 여섯 가지입니다.

이것은 단순한 배경 설명이 아닙니다. 조건 하나하나가 어느 좌표계를 쓸지, 어떤 항을 넣을지를 못 박은 설계 사양입니다.

예를 들어 안테나가 고정형이라는 조건은 설치 방위와 레버암이 상수라는 뜻입니다. 표적이 지표 근처라는 조건은 ECI 가 필요 없다는 뜻입니다. 그리고 역방향 변환도 요구된다는 조건 때문에, 다섯 단계 전부에 역변환을 준비해 두어야 합니다.

02. 단계별 좌표계 선정

여섯 단계, 각 단계에서 왜 그 좌표계를 쓰는가

단계	좌표계	왜 이것인가
0	안테나 - 극좌표	레이다가 물리적으로 잴 수 있는 것이 거리와 두 각도뿐이다
1	안테나 - 직교좌표	회전행렬은 벡터에만 곱할 수 있다.
2	동체 (Body)	안테나 · INS · 무장이 만나는 유일한 공통 기준.
3	로컬 (NED)	배와 함께 가되 흔들리지는 않아 표적의 실제 움직임만 남는다 → 여기서 추적 NED 인 이유 : 동체 FRD 와 축 의미가 같다
4	ECEF	배 위치를 더하려면 지구에 고정된 직교좌표가 필요 - 로컬과 위경도의 다리 역할
5	LLA	전시기 · 통제제어가 지도 위에 표시하는 형식

INS

무게중심에 축을 맞춰 설치.

UV

방향만 있고 거리가 없다.

ECI관성 동역학 전파가 필요할 때만
(위성 · 탄도탄)**ENU**NED 와 정보량이 같다
- 동체 FRD 와 축 의미가 다름

19

발표 대본

02. 실전 설계 · 약 1분 05초

설계에서 중요한 것은 쓸 것을 고르는 것만큼이나 안 쓸 것을 정하는 일입니다.

UV 는 뺐습니다. 방향만 있고 거리가 없어서 표적의 위치를 옮기는 데 쓸 수 없습니다.

ECI 도 뺐습니다. 관성 동역학으로 궤도를 전파할 일이 없기 때문입니다. 위성이나 탄도탄이라면 필요하지만 이번 과제는 아닙니다.

ENU 도 뺐습니다. NED 와 정보량이 완전히 같은데, 동체 좌표계가 FRD 라 셋째 축의 의미를 맞추기에는 NED 가 자연스럽습니다.

INS 는 좌표계라기보다 값의 공급원으로 씁니다. 무게중심에 축을 맞춰 설치했다고 보고, 자세와 위치를 받아 쓰기 만 합니다.

그래서 남은 것이 다섯 단계, 앞 장에서 본 변환 체인입니다.

02. 단계별 변환 안테나 → 동체 → 로컬

① 안테나 극좌표
→ 직교 좌표

표현 변경 (회전 · 이동 없음)

$$\begin{aligned}
 x &= R \cos El \sin Az = 20,000 \times \cos 0^\circ \times \sin 30^\circ = +10,000.00 \text{ m} \\
 y &= R \sin El = 20,000 \times \sin 0^\circ = 0.00 \text{ m} \\
 z &= R \cos El \cos Az = 20,000 \times \cos 0^\circ \times \cos 30^\circ = +17,320.51 \text{ m} \\
 \rightarrow p \text{ 안테나} &= (+10,000.00, 0.00, +17,320.51) \text{ m}
 \end{aligned}$$

② 안테나 → 동체

회전 + 평행이동

$$p \text{ 동체} = Rz(90^\circ) \cdot Ry(0^\circ) \cdot A \cdot p \text{ 안테나} + \text{레버암} \quad (A = Rz(-90^\circ) \cdot Rx(-90^\circ) \text{ 축 맞춤, 아래 } 3 \times 3 = Rz(90^\circ) \cdot Ry(0^\circ) \cdot A)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} +10,000.00 \\ 0.00 \\ +17,320.51 \end{bmatrix} = \begin{bmatrix} +10,000.00 \\ +17,320.51 \\ 0.00 \end{bmatrix} + \begin{bmatrix} -30 \\ 0 \\ -10 \end{bmatrix} = \begin{bmatrix} +9,970.00 \\ +17,320.51 \\ -10.00 \end{bmatrix}$$

$$\rightarrow p \text{ 동체} = (+9,970.00, +17,320.51, -10.00) \text{ m}$$

③ 동체 → 로컬 (NED)

회전만 (원점이 같음)

$$pNED = Rz(45^\circ) \cdot Ry(0^\circ) \cdot Rx(0^\circ) \cdot p \text{ 동체} = Rz(45^\circ) \cdot p \text{ 동체} \quad (\cos 45^\circ = \sin 45^\circ = 0.7071)$$

$$\begin{bmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} +9,970.00 \\ +17,320.51 \\ -10.00 \end{bmatrix} = \begin{bmatrix} -5,197.59 \\ +19,297.30 \\ -10.00 \end{bmatrix}$$

$$\rightarrow pNED = (-5,197.59, +19,297.30, -10.00) \text{ m, 방위 } 105.074^\circ$$

20

발표 대본

02. 실전 설계 · 약 1분 50초

이제 숫자를 직접 따라가 보겠습니다.

첫째, 극좌표를 직교로 바꿉니다. 고각이 0도라 y는 0입니다. x는 2만 곱하기 사인 30도로 정확히 1만 미터. z는 2만 곱하기 코사인 30도로 1만 7320.51 미터, 이것은 1만 루트3입니다.

둘째, 안테나에서 동체로 갑니다. 축 맞춤 A와 설치 방위 90도를 합치면 가운데 3x3 행렬이 나옵니다. 이것을 곱하면 1만, 1만 7320.51, 0이 되고, 여기에 레버암 (-30, 0, -10)을 더하면 동체 좌표 (9970, 17320.51, -10)이 됩니다. z가 -10인 것은 동체의 z축이 아래를 향하기 때문이고, 표적이 무게중심보다 10미터 위에 있다는 뜻입니다.

셋째, 동체에서 NED로 갑니다. 롤과 피치가 0도라 요 45도만 남습니다. 코사인 45도와 사인 45도가 둘 다 0.7071이니, 북 성분은 -5197.59, 동 성분은 19297.30이 나옵니다.

북이 음수인 것이 이상해 보일 수 있는데, 배가 진북에서 시계방향 45도로 돌아 있고 표적이 우현 쪽에 있으니 결과적으로 남동쪽이 됩니다. 방위각으로 환산하면 105.07도입니다.

02. 단계별 변환 로컬 → ECEF → 위경도

④ 로컬 → ECEF

회전 + 평행이동

$$\begin{aligned} \textcircled{1} RN &= a / \sqrt{1 - e^2 \sin^2 \varphi} = 6,378,137 / \sqrt{1 - 0.00669438 \times 0.5935313^2} = 6,385,671.08 \text{ m} \\ P \text{ 배} &= ((RN+h) \cos \varphi \cos \lambda, (RN+h) \cos \varphi \sin \lambda, (RN(1-e^2)+h) \sin \varphi) \quad (\varphi = 36.408^\circ, \lambda = 127.307^\circ, h = 0) \\ &= (-3,114,830.11, +4,087,762.85, +3,764,723.10) \text{ m} \\ \textcircled{2} \text{ Cecef} \leftarrow \text{ned} &= R_z(\lambda) \cdot R_y(-90^\circ - \varphi) \quad (\sin \varphi = 0.5935, \cos \varphi = 0.8048, \sin \lambda = 0.7954, \cos \lambda = -0.6061) \\ \begin{bmatrix} -\sin \varphi \cos \lambda & -\sin \lambda & -\cos \varphi \cos \lambda \\ -\sin \varphi \sin \lambda & \cos \lambda & -\cos \varphi \sin \lambda \\ \cos \varphi & 0 & -\sin \varphi \end{bmatrix} &= \begin{bmatrix} 0.3597 & -0.7954 & 0.4878 \\ -0.4721 & -0.6061 & -0.6401 \\ 0.8048 & 0 & -0.5935 \end{bmatrix} \cdot \begin{bmatrix} -5,197.59 \\ +19,297.30 \\ -10.00 \end{bmatrix} = \begin{bmatrix} -17,223.68 \\ -9,235.66 \\ -4,177.15 \end{bmatrix} \\ \textcircled{3} P \text{ 표적} &= P \text{ 배} + C \cdot p_{\text{NED}} = (-3,132,053.79, +4,078,527.19, +3,760,545.95) \text{ m} \end{aligned}$$

⑤ ECEF → LLA

위도 · 고도 반박 계산

$$\begin{aligned} \text{경도 } \lambda &= \text{atan2}(Y, X) = \text{atan2}(+4,078,527.19, -3,132,053.79) = 127.522008^\circ \quad (X < 0, Y > 0 \rightarrow 2 \text{ 사분면}) \\ \text{위도 } \varphi : p &= \sqrt{X^2 + Y^2} = 5,142,387.09 \text{ m}, \quad \varphi_0 = \arctan(Z/p) = 36.177411^\circ \quad (\text{지구를 구로 본 첫 값}) \\ \tan \varphi &= (Z + e^2 RN \sin \varphi) / p \quad \text{에 } \varphi_0 \text{ 를 넣고 되풀이 :} \\ \varphi_1 &= 36.360167^\circ \rightarrow \varphi_2 = 36.360964^\circ \rightarrow \varphi_3 = 36.360967^\circ \rightarrow \varphi_4 = 36.360967176^\circ \quad (\text{변화 } < 1e-7^\circ, \text{ 멈춤}) \\ \text{고도 } h &= p / \cos \varphi - RN = 6,385,695.57 - 6,385,654.29 = 41.2824 \text{ m} \\ \rightarrow \text{표적 LLA} &= (36.360967176^\circ \text{N}, 127.522008441^\circ \text{E}, 41.2824 \text{ m}) \end{aligned}$$

표적 LLA = 36.360967176°N 127.522008441°E 고도 41.2824 m

21

발표 대본

02. 실전 설계 · 약 1분 40초

넷째 단계, NED 에서 ECEF 로 갑니다.

먼저 배의 ECEF 위치부터 구해야 합니다. 배의 위도 36.408도를 넣으면 그 위도에서의 곡률반경 RN 이 6,385,671 미터로 나오고, 이것을 공식에 넣으면 배의 ECEF 좌표가 나옵니다.

그다음 NED 벡터를 지구 축에 맞춰 돌립니다. 가운데 3×3 이 경도와 위도로 만들어진 회전행렬이고, 이것을 NED 에 곱하면 (-17223.68, -9235.66, -4177.15) 가 나옵니다. 여기에 배의 ECEF 위치를 더한 것이 표적의 ECEF 입니다.

값이 갑자기 3백만 미터대로 뛰는데, 놀랄 일이 아닙니다. 원점이 지구 중심이니까 지구 반지름만큼이 기본으로 깔리는 것입니다.

다섯째, ECEF 를 위경도로 되돌립니다. 경도는 Y 와 X 의 아크탄젠트로 바로 나옵니다. X 가 음수이고 Y 가 양수라 2사분면이고, 127.522008도입니다. 위도는 앞서 말씀드린 순환 구조라 반복으로 풀니다.

최종 결과가 북위 36.360967도, 동경 127.522008도, 고도 41.28 미터입니다.

02. 역변환

역변환은 새로울 게 없다

- ① 회전은 전치 (transpose) ② 평행이동은 빼기 ③ 순서는 거꾸로

표적 LLA (36.360967176°N, 127.522008441°E, 41.2824 m)

→ ECEF	(-3,132,053.79, +4,078,527.19, +3,760,545.95) m	// 정변환과 같은 공식
→ 로컬 NED	$C_{ecef \leftarrow ned}^T \times (\text{표적} - \text{배}) = (-5,197.59, +19,297.30, -10.00) \text{ m}$	// 전치 + 빼기
→ 동체	$C_{ned \leftarrow body}^T \times p_{NED} = (+9,970.00, +17,320.51, -10.00) \text{ m}$	// 전치
→ 안테나	$C_{body \leftarrow ant}^T \times (p_{\text{동체}} - \text{레버암}) = (+10,000.00, 0.00, +17,320.51) \text{ m}$	// 전치 + 빼기
→ 극좌표	$R = \sqrt{x^2 + y^2 + z^2}, \quad Az = \text{atan2}(x, z), \quad El = \text{atan2}(y, \sqrt{x^2 + z^2})$	

결과 : R = 20,000.000000 m Az = 30.000000° El = 0.000000°

22

발표 대본

02. 실전 설계 · 약 1분 10초

과제 조건에 역방향 변환도 요구되어 있습니다. 그런데 역변환은 새로 만들 것이 거의 없습니다.

규칙은 세 가지입니다. 회전은 전치. 앞에서 R 전치가 곧 역행렬이라고 했으니 그대로 씁니다. 평행이동은 빼기. 더 했던 것을 빼면 됩니다. 그리고 순서는 거꾸로.

표를 따라가 보겠습니다. 위경도에서 ECEF 로 갈 때는 정변환과 똑같은 공식을 씁니다. ECEF 에서 NED 로 갈 때는 배의 위치를 먼저 빼고 전치 행렬을 곱합니다. NED 에서 동체는 원점이 같으니 전치만 곱하면 됩니다. 동체에서 안테나는 레버암을 먼저 빼고 전치를 곱합니다.

마지막으로 직교좌표를 극좌표로 되돌리면 R 이 2만 미터, 방위 30도, 고각 0도. 처음 입력값이 그대로 돌아왔습니다.

C언어로 구현

03

- 1 회전행렬 - 공식과 코드
- 2 안테나 → 동체 - 축 맞춤과 회전 순서
- 3 안테나 극좌표 → 직교 → 동체
- 4 동체 → NED → ECEF
- 5 ECEF ↔ LLA
- 6 역변환
- 7 실행 결과와 UI 확인

23

발표 대본

03. C언어로 구현 · 약 30초

마지막으로 C 구현입니다.

회전행렬을 코드로 어떻게 옮기는지부터 시작하겠습니다. 그다음 앞에서 가장 헷갈리기 쉬웠던 안테나에서 동체로 가는 부분을 따로 자세히 보고, 나머지 단계는 공식과 코드를 나란히 놓고 확인하겠습니다. 마지막에 실행 결과가 손계산과 맞는지 보겠습니다.

03. 회전행렬 - 공식과 코드

공식의 3×3 을 그대로 배열 칸에 옮겨 담는다. 0 인 칸은 아예 건드리지 않는다.

공식

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

C 구현 f_RotZ()

```
static matrix f_RotZ(const FLOAT64 dAng)
{
    matrix stR;
    Matrix_initialize(&stR, 3, 3); // 3x3 으로 잡고 전부 0
    stR.e[0][0] = cos(dAng); stR.e[0][1] = -sin(dAng);
    stR.e[1][0] = sin(dAng); stR.e[1][1] = cos(dAng);
    stR.e[2][2] = 1.0;
    return stR;
}
```

행렬 칸 ↔ 배열 첨자는 1 : 1

e[0][0] = cos θ e[0][1] = -sin θ e[1][0] = sin θ e[1][1] = cos θ e[2][2] = 1 나머지 네 칸은 0 그대로
f_RotX · f_RotY 도 같은 방식이다. 다만 Ry 만 부호가 반대라 e[0][2] = +sin, e[2][0] = -sin 이다 (p.6 공식 그대로).

곱하는 순서가 결과를 바꾼다

R₁ · R₂ ≠ R₂ · R₁ — 회전에는 교환법칙이 없다. 오른쪽 행렬이 벡터에 먼저 걸린다.
각 회전은 자기 축이 놓인 좌표계에 도달한 뒤에 걸어야 한다.
순서를 바꿔도 에러가 아니라 조용히 다른 값이 나온다 (p.25 · p.27).

24

발표 대본

03. C언어로 구현 · 약 1분 40초

회전행렬 구현은 사실상 옮겨 적기입니다.

Matrix_initialize 로 3×3 을 잡으면 아홉 칸이 전부 0 으로 채워집니다. 그러면 공식에서 0 이 아닌 칸만 채우면 됩니다. Rz 는 다섯 칸입니다. e[0][0] 에 코사인, e[0][1] 에 마이너스 사인, e[1][0] 에 사인, e[1][1] 에 코사인, 그리고 e[2][2] 에 1. 나머지 네 칸은 0 인 채로 두면 됩니다. 공식의 칸 위치와 배열 첨자가 1대1 로 대응합니다.

f_RotX 와 f_RotY 도 같은 방식인데, Ry 만 부호 배치가 반대입니다. e[0][2] 가 플러스 사인이고 e[2][0] 이 마이너스 사인입니다. 세 축 중에 여기만 다르니 코드를 보실 때 주의해서 봐 주시면 됩니다.

그리고 이 장에서 꼭 기억하실 것이 아래 상자입니다. 회전에는 교환법칙이 없습니다. 곱하는 순서를 바꾸면 결과가 달라지는데, 문제는 컴파일 에러도 런타임 에러도 나지 않는다는 점입니다. 그냥 조용히 다른 값이 나옵니다. 다음 장에서 실제로 그런 자리가 어디인지 보겠습니다.

03. 안테나 → 동체 회전이 세 번인 이유

안테나 축을 동체 축 이름으로 바꾸고 (A), 그 위에 실제 설치 자세를 얹는다 (Ry · Rz).

① A — 축 이름 재배치 (설치와 무관한 상수)

안테나 x (왼쪽) → 동체 -y (좌현)
 안테나 y (위) → 동체 -z (위)
 안테나 z (보어사이트) → 동체 +x (선수)
 돌리는 게 아니라 (좌, 위, 정면) 을
 (정면, 우, 아래) 순서로 다시 적는 것. 그래서 상수.

② ③ 설치 자세 — 보어사이트 추적 (방위 90°, 백틸트 20° 에)

A · p 선수 고각 0°
 Ry(τ) · A · p 선수에서 위로 고각 20°
 Rz(ψ) · Ry(τ) · A · p 우현에서 위로 고각 20° ← 실제 설치
 ψ = 설치 방위 (벡터에서 시계방향), τ = 안테나 면이 돌린 각

왜 이 순서인가 — 회전은 자기 축이 놓인 좌표계에 도달한 뒤에 걸린다

p_body = Rz(ψ) · Ry(τ) · A · p_ant + t_lever 맨 오른쪽 A 가 벡터에 가장 먼저 걸린다
 Rz(ψ) 는 동체 z(아래) 둘레 회전이다. A 보다 먼저 걸리면 벡터가 아직 안테나 좌표여서, 동체의 '아래' 가 아니라 보어사이트 축 둘레로 돌아 버린다.

순서를 바꾸면

A · Rz(ψ) → (17,320.51, 0, -10,000) — 표적이 상공 10 km
 Ry(τ) · Rz(ψ) → 우현 설치에서 백틸트가 통째로 사라진다
 회전축과 보어사이트가 겹쳐 Ry 가 아무 일도 하지 않기 때문

이번 과제에서는 드러나지 않는다

백틸트가 0° 라 Ry = I. 순서를 바꿔도 결과가 같다.
 왕복오차도 정 · 역이 같은 행렬을 써서 이 오류를 못 잡는다.
 백틸트를 주는 순간 조용히 틀리는 자리다.

25

발표 대본

03. C언어로 구현 · 약 2분 45초

안테나에서 동체로 갈 때 왜 회전이 세 번인지, 이 장에서 정리하겠습니다.

왼쪽 위가 첫 번째인 A 입니다. 이것은 사실 돌리는 것이 아닙니다. 안테나 좌표계는 x 가 왼쪽, y 가 위, z 가 보어사이트 순서로 적습니다. 동체는 x 가 선수, y 가 우현, z 가 아래 순서로 적습니다. 표를 보시면 안테나의 보어사이트가 동체의 선수 자리로, 안테나의 왼쪽이 좌현으로, 안테나의 위가 그대로 위로 갑니다. 물리적으로는 아무것도 움직이지 않았습니다. 같은 방향을 다른 순서로 다시 적었을 뿐입니다. 그래서 A 는 설치와 무관한 상수입니다.

오른쪽 위가 그다음입니다. A 만 적용하면 보어사이트가 선수를 향합니다. 여기에 백틸트를 걸면 선수에서 위로 돌리고, 마지막에 설치 방위를 걸면 우현 방향에서 위로 들린 실제 설치 자세가 됩니다.

가운데가 왜 이 순서여야 하는가입니다. Rz 는 동체의 z 축, 즉 아래 축 둘레로 도는 회전입니다. 그런데 A 보다 먼저 걸리면 벡터가 아직 안테나 좌표에 있습니다. 안테나의 z 는 보어사이트이니, 동체의 아래 축이 아니라 보어사이트 축 둘레로 돌아 버립니다. 그래서 A 가 반드시 먼저 걸려야 합니다.

아래 왼쪽이 실제로 순서를 바꿨을 때입니다. A 와 Rz 를 바꾸면 표적이 상공 10 킬로미터에 뜹니다. Ry 와 Rz 를 바꾸면 더 무서운데, 우현 설치에서는 백틸트가 통째로 사라집니다. 회전축과 보어사이트가 겹쳐서 Ry 가 아무 일도 하지 않기 때문입니다.

그런데 아래 오른쪽이 진짜 함정입니다. 이번 과제는 백틸트가 0도라 Ry 가 단위행렬입니다. 순서를 바꿔도 결과가 똑같이 나옵니다. 왕복오차 검증도 정변환과 역변환이 같은 행렬을 쓰기 때문에 이 오류를 잡지 못합니다. 백틸트를 주는 순간 조용히 틀리기 시작하는 자리입니다.

03. 공식과 코드 ① 안테나 극좌표 → 직교 → 동체

①
극좌표 → 직교
표현만 바꿈

$$\begin{aligned}x &= R \cos(EI) \sin(Az) \\y &= R \sin(EI) \\z &= R \cos(EI) \cos(Az)\end{aligned}$$

②
안테나 → 동체
회전 + 평행이동

$$p_{body} = R_z(\psi_{mount}) R_y(\tau_{tilt}) A p_{ant} + t_{lever}$$

$$A = R_z(-90^\circ) R_x(-90^\circ)$$

$$A = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

C 구현

```
ST_Vec3 f_PolarToXYZ(const ST_Polar stPolar)
{
    stXYZ.dX = stPolar.dRange * cos(stPolar.dEI)
              * sin(stPolar.dAz);
    stXYZ.dY = stPolar.dRange * sin(stPolar.dEI);
    stXYZ.dZ = stPolar.dRange * cos(stPolar.dEI)
              * cos(stPolar.dAz);
}
```

```
static matrix f_RotAntToBody(const ST_Mount m)
{
    matrix stRz = f_RotZ(m.dAz); // 설치 방위
    matrix stRy = f_RotY(m.dTilt); // 백틸트
    matrix stAxis = f_RotAntAxisToBody(); // 축 맞춤 A
    matrix stMnt = Matrix_Product2(&stRz, &stRy);
    return Matrix_Product2(&stMnt, &stAxis);
}
```

```
ST_Vec3 f_AntToBody(...)
{
    stRot = Matrix_Product2(&stR, &stP); // R·p
    stOut = Matrix_Add(&stRot, &stOff); // + 레버암
}
```

이번 과제 값

$p_{ant} = (+10,000.00, 0.00, +17,320.51) \text{ m}$ → $p_{body} = (+9,970.00, +17,320.51, -10.00) \text{ m}$ (A 는 설치 방위 0° 기준 · 축과 순서는 p.25)

26

발표 대본

03. C언어로 구현 · 약 1분 10초

이제 단계별로 공식과 코드를 나란히 놓고 보겠습니다.

첫 번째, 극좌표에서 직교로. 왼쪽 공식 세 줄이 오른쪽 코드 세 줄과 그대로 대응합니다. 별도 설명이 필요 없을 정도입니다.

두 번째, 안테나에서 동체로. 식은 R_z 곱하기 R_y 곱하기 A 곱하기 p_{ant} , 더하기 레버암입니다. 코드를 보시면 설치 방위로 R_z 를, 백틸트로 R_y 를, 축 맞춤으로 A 를 각각 만들고, Product2 로 R_z 와 R_y 를 먼저 곱한 다음 거기에 A 를 곱합니다. 결과적으로 A 가 맨 오른쪽에 남아서 벡터에 가장 먼저 걸립니다. 식의 순서와 코드의 순서가 정확히 같습니다.

아래 띠가 이번 과제 값인데, 앞에서 손으로 계산한 것과 같은 숫자가 나옵니다.

03. 공식과 코드 ② 동체 → NED → ECEF

	공식	C 구현
③ 동체 → NED 회전만 (원점 같음)	$p_{NED} = R_z(yaw) R_y(pitch) R_x(roll) p_{body}$	<pre>static matrix f_RotBodyToNed(const ST_Attitude a) { matrix stRz = f_RotZ(a.dYaw); matrix stRy = f_RotY(a.dPitch); matrix stRx = f_RotX(a.dRoll); return Matrix_Product3(&stRz, &stRy, &stRx); } // Product3 = (Rz·Ry)·Rx → Rx 가 먼저 걸림</pre>
④ NED → ECEF 회전 + 평행이동	$p_{ECEF} = R_z(\lambda) R_y(-90^\circ - \varphi) p_{NED} + P_{ship}$	<pre>static matrix f_RotNedToEcef(const ST_Lla o) { matrix stRz = f_RotZ(o.dLon); matrix stRy = f_RotY(-PI / 2.0 - o.dLat); return Matrix_Product2(&stRz, &stRy); } ST_Vec3 f_NedToEcef(const ST_Lla o, const ST_Vec3 n) { stRot = Matrix_Product2(&stRz, &stRy); stShip = f_ToMat(f_LlaToEcef(o)); // 함선 ECEF 를 함수가 안에서 구한다 stOut = Matrix_Add(&stRot, &stShip); }</pre>
이번 과제 값 $p_{NED} = (-5,197.59, +19,297.30, -10.00) \text{ m}$ $p_{ECEF} = (-3,132,053.79, +4,078,527.19, +3,760,545.95) \text{ m}$		

27

발표 대본

03. C언어로 구현 · 약 1분 25초

세 번째, 동체에서 NED 로 갑니다. 원점이 같아서 평행이동이 없고 회전만 걸립니다.

코드에서 요, 피치, 롤로 각각 회전행렬을 만들고 Product3 로 한 번에 곱하는데, Product3 가 왼쪽부터 묶기 때문에 결과적으로 롤이 벡터에 가장 먼저 걸립니다. 이것이 표준 ZYX 순서이고, 순서를 바꾸면 값이 달라지니 주의해야 합니다.

네 번째, NED 에서 ECEF 로 갑니다. 회전은 경도로 만든 Rz 와, 마이너스 90도 빼기 위도로 만든 Ry 두 개입니다. 여기에 배의 ECEF 위치를 더하는데, 코드를 보시면 그 배의 위치를 함수가 안에서 직접 구합니다. 배의 위경도를 받아서 f_LlaToEcef 를 호출하는 방식이라, 호출하는 쪽에서 미리 계산해 넘길 필요가 없습니다.

아래 띠를 보시면 NED 에서 ECEF 로 넘어가는 순간 자릿수가 확 커집니다. 원점이 지구 중심이기 때문입니다.

03. 공식과 코드 ③ ECEF ↔ LLA

	공식	C 구현
⑤ LLA → ECEF 한 번에 나온다	$N = a / \sqrt{1 - e^2 \sin^2 \varphi}$ $X = (N + h) \cos \varphi \cos \lambda$ $Y = (N + h) \cos \varphi \sin \lambda$ $Z = (N(1 - e^2) + h) \sin \varphi$	<pre> dN = WGS84_A / sqrt(1.0 - WGS84_E2 * sin(dLat) * sin(dLat)); stEcef.dX = (dN + dAlt) * cos(dLat) * cos(dLon); stEcef.dY = (dN + dAlt) * cos(dLat) * sin(dLon); stEcef.dZ = (dN * (1.0 - WGS84_E2) + dAlt) * sin(dLat); // z 에만 (1 - e^2) - 극 쪽으로 늘린 만큼 </pre>
⑥ ECEF → LLA 위도 · 고도 반복	$\lambda = \text{atan2}(Y, X) \quad p = \sqrt{X^2 + Y^2}$ $\varphi_0 = \text{atan2}(Z, p) \quad h_0 = 0$ $N_k = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi_k}} \quad h = \frac{p}{\cos \varphi_k} - N_k$ $\varphi_{k+1} = \text{atan2}(Z(N_k + h), p(N_k(1 - e^2) + h))$	<pre> stLla.dLon = atan2(stEcef.dY, stEcef.dX); dLat0 = atan2(stEcef.dZ, dP); dAlt0 = 0.0; for (i = 0; i < LLA_ITER_MAX; i++) { dN = WGS84_A / sqrt(1.0 - WGS84_E2 * sin(dLat0) * sin(dLat0)); dAlt = dP / cos(dLat0) - dN; dLat = atan2(dZ * (dN + dAlt), dP * (dN * (1.0 - WGS84_E2) + dAlt)); if (fabs(dLat - dLat0) < LLA_TOL_LAT && fabs(dAlt - dAlt0) < LLA_TOL_ALT) break; dLat0 = dLat; dAlt0 = dAlt; } </pre>

반복 결과 위도 [deg]
 초기 36.177411 → ① 36.361399 → ② 36.360966 → ③ 36.3609672 → ④⑤ 36.360967176 → ⑥ 변화 2e-16, break (고도 41.2824 m · 최대 10 회 · 허용오차 1e-12 rad)

28

발표 대본

03. C언어로 구현 · 약 1분 35초

다섯 번째, LLA 와 ECEF 사이입니다.

LLA 에서 ECEF 로는 공식에 넣으면 바로 나옵니다. 코드도 네 줄입니다. 여기서 눈여겨보실 것은 z 성분에만 1 빼기 e 제곱이 붙는다는 점입니다. 지구가 극 쪽으로 늘린 만큼 z 를 줄여 주는 것이고, x 와 y 에는 붙지 않습니다.

반대 방향이 오늘 유일하게 한 번에 풀리지 않는 단계입니다. 경도는 아크탄젠트로 바로 나오지만, 위도는 곡률반경 RN 이 있어야 구할 수 있고 RN 은 위도가 있어야 구할 수 있습니다. 그래서 초기값으로 지구를 그냥 구로 봤을 때의 위도를 넣고, 위도와 고도를 번갈아 계산하면서 값이 더 안 변할 때까지 반복합니다.

아래가 실제 수렴 과정입니다. 초기값 36.177 에서 시작해서 첫 번째 반복에 36.3614, 세 번째에 이미 소수점 일곱 자리가 맞고, 여섯 번째에서 변화가 10의 마이너스 16승이 되어 빠져나옵니다. 최대 열 번으로 상한을 걸어 두었지만 실제로는 대여섯 번이면 끝납니다.

03. 공식과 코드 ④ 역변환

새로 만드는 공식이 없다. 회전은 전치, 이동은 빼기, 그리고 순서를 거꾸로.

$$\begin{aligned} \text{forward } p' &= R p + t \\ \text{inverse } p &= R^T (p' - t) \end{aligned}$$

순서까지 뒤집어야 한다

레버암은 동체 좌표계에서 잰 값이다. 그래서 동체 좌표에서 먼저 빼고, 그다음에 안테나 축으로 되돌린다. 두 줄을 바꾸면 레버암이 엉뚱한 좌표계에서 빠져 결과가 틀어진다.

정변환 **f_AntToBody**

```
stR = f_RotAntToBody(stMount);
stRot = Matrix_Product2(&stR, &stP); // ① 회전
stOut = Matrix_Add(&stRot, &stOff); // ② 더하기
```

역변환 **f_BodyToAnt**

```
stRt = Matrix_Transpose(&stR); // 전치
stDiff = Matrix_Subtract(&stP, &stOff); // ① 빼기
stOut = Matrix_Product2(&stRt, &stDiff); // ② 회전
```

직교 → 극좌표 **f_XyzToPolar**

$$\begin{aligned} R &= \sqrt{x^2 + y^2 + z^2} \\ Az &= \text{atan2}(x, z) \\ El &= \text{atan2}(y, \sqrt{x^2 + z^2}) \end{aligned}$$

```
dXz = sqrt(dX*dX + dZ*dZ);
dRange = sqrt(dXz*dXz + dY*dY);
dAz = atan2(stXyz.dX, stXyz.dZ);
dEl = atan2(stXyz.dY, dXz);
```

29

발표 대본

03. C언어로 구현 · 약 1분 15초

역변환 구현입니다. 앞에서 본 규칙 세 가지가 코드로도 그대로 나타납니다.

위쪽 식을 보시면 정변환이 $p' = R p + t$ 이고, 역변환은 $p = R^T (p' - t)$ 입니다.

아래 두 코드를 비교해 보시면, 정변환은 회전하고 더합니다. 역변환은 전치를 만들고, 빼고, 회전합니다.

여기서 놓치기 쉬운 것이 순서입니다. 오른쪽 위 상자에 있는데, 레버암은 동체 좌표계에서 잰 값입니다. 그래서 동체 좌표 상태에서 먼저 빼고, 그다음에 안테나 축으로 되돌려야 합니다. 두 줄을 바꿔서 안테나 축으로 먼저 돌린 다음 빼면, 레버암이 엉뚱한 좌표계에서 빠져 결과가 틀어집니다.

아래는 직교좌표를 극좌표로 되돌리는 부분입니다. 거리는 세 성분의 제곱합의 제곱근이고, 방위각과 고각은 아크 탄젠트로 구합니다.

03. 실행 결과

radar_coord.exe 를 그대로 돌린 화면 . 발표자료 p.20 ~ p.22 의 손계산과 전 단계가 일치한다 .

```
[ 입력 ]
측장값   R = 20000.0 m, Az = 30.000 deg, El = 0.000 deg
물렛줄 위치 lat = 36.408000 deg, lon = 127.307000 deg, alt = 0.0 m
물렛줄 자세 (r, y, p) = (0.000, 45.000, 0.000) deg
안테나 설치 az = 90.000 deg, tilt = 0.000 deg, offset = (-30.0, 0.0, -10.0) m

[ 정변환 ]
1) 극좌표 -> 안테나 직교   10000.0000    0.0000   17320.5081 [m]
2) 안테나 -> 동체        9970.0000   17320.5081   -10.0000 [m]
3) 동체 -> NED           -5197.5941   19297.3033   -10.0000 [m]
4) NED -> ECEF          -3132053.7872  4078527.1919  3760545.9526 [m]
합선 ECEF               -3114830.1107  4087762.8525  3764723.0978 [m]
5) ECEF -> LLA
   lat = 36.360967176 deg, lon = 127.522008441 deg, alt = 41.2824 m

[ 역변환 ]      ( 정변환의 값을 그대로 되짚어 온다 )
2) ECEF -> NED           -5197.5941   19297.3033   -10.0000 [m]
3) NED -> 동체          9970.0000   17320.5081   -10.0000 [m]
4) 동체 -> 안테나 직교   10000.0000    0.0000   17320.5081 [m]
5) R = 20000.000000 m, Az = 30.000000000 deg, El = 0.000000000 deg
   입력과 차이 dR = 6.8e-10 m, dAz = 6.8e-13 deg, dEl = 3.8e-12 deg
```

ECEF 에서 자릿수가 켜다

표적까지 20 km 인데 좌표는 3,132 km 다 . 원점이 지구 중심이라서 .
float 32bit 은 이 크기에서 눈금이 0.25 m — 미터 이하가 날아간다 . 전 구간 double 을 쓰는 이유다 .

과제 최종 출력 2 가지

- ① 36.360967 °N, 127.522008 °E, 41.3 m
- ② R 20000.0 m, Az 30.000°, El 0.000°

고도가 41.3 m 인 이유

표적은 수평면에서 10 m 위지만 , 20 km 앞이라 지구 표면이 31.3 m 아래로 내려간다 .



30

발표 대본

03. C언어로 구현 · 약 2분 20초

실제로 radar_coord.exe 를 돌린 화면입니다.

위쪽이 입력이고, 그 아래가 정변환 다섯 단계입니다. 1번부터 5번까지 값이 앞에서 손으로 계산한 것과 전부 일치합니다. 그 아래 역변환은 이 값을 그대로 되짚어 온 것인데, 마지막 줄을 보시면 입력과의 차이가 거리 기준 10의 마이너스 10승 미터 수준입니다.

오른쪽 위를 봐 주십시오. 4번 단계에서 좌표가 갑자기 3천 킬로미터대로 됩니다. 표적까지의 거리는 20 킬로미터 인데 좌표값은 3,132 킬로미터입니다. 원점이 지구 중심이기 때문입니다.

이것이 왜 중요하냐면, 이 크기의 숫자를 32비트 float 으로 담으면 표현할 수 있는 눈금 간격이 0.25 미터가 됩니다. 미터 이하가 통째로 날아간다는 뜻입니다. 그래서 이 라이브러리는 전 구간을 double, 코드에서는 FLOAT64 로 씁니다.

가운데는 과제가 요구한 최종 출력 두 가지입니다. 표적의 위경도와 고도, 그리고 역변환으로 되돌린 안테나 극좌표 입니다.

아래가 고도 이야기인데, 조금 의아하실 수 있습니다. 표적은 배의 수평면에서 10 미터 위인데 왜 고도가 41.3 미터로 나오느냐. 그림을 보시면, 파란 선이 수평면을 따라 20 킬로미터를 직진한 선입니다. 그런데 지구는 휘어 있으니 지구 표면은 그 직선에서 점점 멀어집니다. 20 킬로미터 지점에서 그 차이가 31.3 미터입니다. 표적은 수평면보다 10 미터 위에 있으니, 지구 타원체면 기준으로는 10 더하기 31.3, 약 41.3 미터가 되는 것입니다.

03. UI 화면에서 확인

CoordUI 는 같은 CoordCore 를 호출한다 . 입력을 바꾸면 단계별 값이 그 자리에서 다시 계산된다 .

CoordFrames

측정값 (안테나 기준)

R [m] 20000.0 Az [deg] 30.000 El [deg] 0.000

플랫폼 위치

lat [deg] 36.408000 lon [deg] 127.307000 alt [m] 0.0

플랫폼 자세 [deg]

roll 0.0 pitch 0.0 yaw 45.0

안테나 설치

az [deg] 90.000 tilt [deg] 0.000

offset x / y / z [m] -30.0 0.0 -10.0

예제값 복원

입력 각도도 / 변환 계산 라디안, m

단계별 결과 (길이 m, 각도 deg)

단계	x / N / X	y / E / Y	z / D / Z
정변환 1) 극좌표 -> 안테나 직교 (x, y, z)	10000.0000	0.0000	17320.5081
정변환 2) 안테나 -> 동체 (x, y, z)	9970.0000	17320.5081	-10.0000
정변환 3) 동체 -> NED (N, E, D)	-5197.5941	19297.3033	-10.0000
정변환 4) NED -> ECEF (X, Y, Z)	-3132053.7872	4078527.1919	3760545.9526
합성 ECEF (X, Y, Z)	-3114830.1107	4087762.8525	3764723.0978
정변환 5) ECEF -> LLA (lat, lon, alt)	36.360967	127.522008	41.282403
역변환 1) LLA -> ECEF (X, Y, Z)	-3132053.7872	4078527.1919	3760545.9526
역변환 2) ECEF -> NED (N, E, D)	-5197.5941	19297.3033	-10.0000
역변환 3) NED -> 동체 (x, y, z)	9970.0000	17320.5081	-10.0000
역변환 4) 동체 -> 안테나 직교 (x, y, z)	10000.0000	0.0000	17320.5081
역변환 5) 안테나 직교 -> 극좌표 (x, y, z)	20000.000000	30.000000	0.000000

최종 출력 (통제제어 전달 값)

1) 표적 위 / 경 / 고도 lat = 36.360967176 deg, lon = 127.522008441 deg, alt = 41.2824 m

2) 안테나 극좌표 R = 20000.0000 m, Az = 30.000000 deg, El = 0.000000 deg

왕복오차 dr = 6.8e-10 m, dAz = 6.8e-13 deg, dEl = 3.8e-12 deg

Close

표에 찍힌 값이 발표자료 p.20 ~ p.22 · 앞 장 콘솔 출력과 모두 같다 .

① 입력

측정값 : 플랫폼 위치 · 자세 · 안테나 설치값 그대로 넣는다 .
roll / pitch / yaw 는 슬라이더로도 바꾼다 .

② 단계별 결과

정변환 5 단계와 역변환 5 단계가 한 표에 나온다 .
콘솔 출력과 값이 같다 .

③ 최종 출력

표적 위 / 경 / 고도와 안테나 극좌표 ,
그리고 왕복오차까지 한 눈에 확인된다 .

31

발표 대본

03. C언어로 구현 · 약 1분 00초

마지막으로 UI 화면입니다.

같은 CoordCore 라이브러리를 MFC 쪽에서도 그대로 호출합니다. 계산 로직이 한 곳에만 있고 콘솔과 UI 가 같은 함수를 부르는 구조입니다.

왼쪽에서 측정값, 플랫폼 위치와 자세, 안테나 설치값을 입력합니다. 롤, 피치, 요는 슬라이더로도 바꿀 수 있어서 배가 흔들릴 때 결과가 어떻게 변하는지 바로 확인할 수 있습니다.

가운데 표가 정변환 다섯 단계와 역변환 다섯 단계를 한 번에 보여주고, 그 아래가 최종 출력과 왕복오차입니다.

표에 찍힌 값이 앞에서 본 손계산, 그리고 콘솔 출력과 모두 같습니다.

이상으로 발표를 마치겠습니다. 감사합니다.